Ordering & Management System for Educational Escape Room Games

Yacob Hiben
Nathanel Tekalign

CIS248 Advanced Application Development

July 25, 2025

The Problem

- Poor inventory and quest management systems.
- Cluttered inventories, unclear item functions, and disorganized quest logs.
- No proper sorting tools, filters, or legends, players are left guessing what to keep, sell, or pursue.

Why this solution?

- Combines the excitement of escape room challenges with educational goals
- Designed for clarity, ease of use, and meaningful engagement.
- Players can explore available games, play demos, and customize their experiences with just a few clicks.
- Simplifies the ordering process, supports learning through problem-solving and collaboration, and provides a smooth, frustration-free user experience.

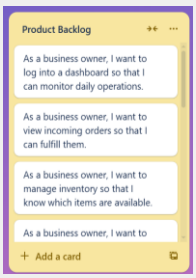
Requirements

- User login & registration
- Game catalog browsing
- Ordering & payment integration
- Owner/admin dashboards
- Real-time updates and reporting

System Information

- Frontend: React.js
- Backend: Node.js & Express
- Database: JSON-based file storage (prototype phase)
- Other tools: Visual Studio Code, draw.io / Visio for diagrams

Product backlog



Sprint 1

Backlog ID	User Stories	Initial Estimate Day 0
1	Register	4
2	Log in	3
3	Browse	3
4	Customize	4
5	Pay	6
6	Notify	3
7	Track	4
8	Demo	4
9	FAQ	3
10	Feedback	4
Remaining Effort		38

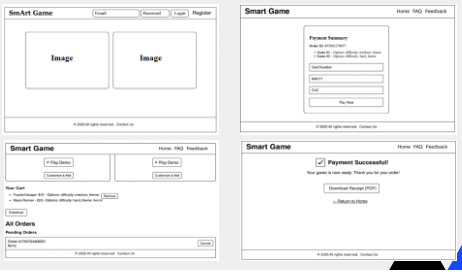
Sprint 2

Backlog ID	User Stories	Initial Estimate Day 0
1	Sprint 1 complete	6
2	Owner/Admin Log in	4
3	Order list	4
4	Inventory mngmt	4
5	Track status	3
6	Edit listing	5
7	Download reports	3
8	Update price	3
9	Manage user	4
10	Monitor system	3
11	Racing alert	4
Remaining Effort		43

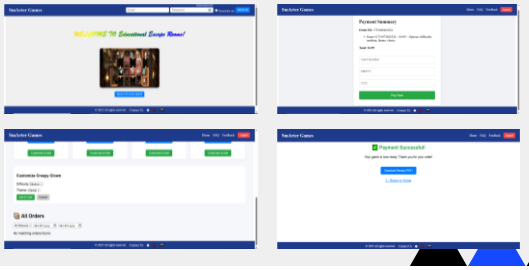
Last sprint backlog

SN	Role	User Story	Priority	Est. (hrs)	Status
1	All	Refining dashboards	High	6	Completed
2	All	Cleaning codes	High	6	Completed
3	All	Testing app	High	6	Completed
4	All	Documentation	High	6	Completed

GUI design & wireframes



Screenshots of the system



Code example #1

- Configures the Express backend to run on port 3001:

```
const PORT = process.env.PORT || 3001;

app.use(cors());
app.use(express.json());
app.use(bodyParser.urlencoded{ extended: false });

try { newDB(); }
catch {
  writeDB({ users: [], games: [
    { id: 1, title: "Puzzle Escape", price: 15 },
    { id: 2, title: "Haze Runner", price: 20 }
  ], orders: [] });
}

app.use("/api/auth", authRoutes);
app.use("/api/games", gameRoutes);
app.use("/api/orders", (req, res, next) => {
  req.io = io;
  next();
},orderRoutes);
app.use("/api/feedback", feedbackRoutes);
app.use("/api/owner", ownerRoutes);
io.on("connection", socket => {
  console.log("Socket connected:", socket.id);
});
app.use("/api/receipt", receiptRoutes);
app.use("/api/users", userRoutes);
app.use("/api/logs", logRoutes);

server.listen(PORT, () => console.log(`Backend + Socket.IO running on port ${PORT}`));
```

- Sets up a centralized, secure way to communicate between React frontend and backend API:

```
// Create an Axios instance pointed at backend's API
export const api = axios.create{
  baseURL: process.env.REACT_APP_API_URL || "http://localhost:3001/api",
});

// Automatically add the JWT token (if it exists) to every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Intercept responses to centrally handle authorization errors
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location = "/";  // Redirect to login
    }
    return Promise.reject(error);
  }
);
```

Code example #2

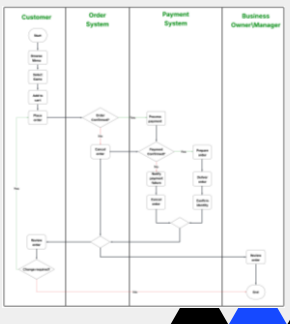
```
const addToCart = (game, options = {}) => {
  const orderItem = { id: Date.now(), game, options };
  setCart((prev) => [...prev, orderItem]);
  setSelected(null);
};

const removeFromCart = (id) => {
  setCart((prev) => prev.filter((item) => item.id !== id));
};

const proceedToPayment = () => {
  if (cart.length === 0) return alert("Cart is empty!");
  api
    .post("/orders", { cart })
    .then((res) => {
      setOrders((prev) => [...prev, res.data.order]);
      setCart([]);
      navigate("/payment", { state: { order: res.data.order } });
    })
    .catch((err) => {
      console.error("Failed to place order:", err);
      alert("Checkout failed.");
    });
};
```

```
const handleStatusChange = (orderId, newStatus) => {
  api.put(`/orders/${orderId}`, { status: newStatus },
    headers: { Authorization: `Bearer ${token}` }
  )
    .then(() => {
      setOrders(prev =>
        prev.map(o =>
          o.id === orderId ? { ...o, status: newStatus } : o
        )
      );
    })
    .catch(err => {
      console.error("Failed to update status:", err);
      alert("Error updating order status.");
    });
};
```

```
function downloadReport() {
  api.get("/owner/reports/sales", { responseType: "blob" })
    .then(r => {
      const url = window.URL.createObjectURL(new Blob([r.data], { type: "text/csv" }));
      const link = document.createElement("a");
      link.href = url;
      link.setAttribute("download", "sales_report.csv");
      document.body.appendChild(link);
      link.click();
      link.remove();
    }
    .catch(err => {
      console.error("Download error:", err);
      setReport("Report download failed. Please try again.");
    });
}
```

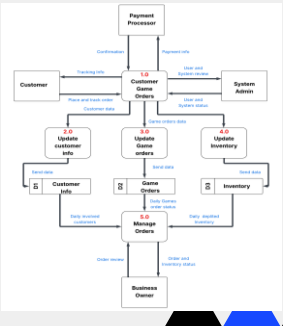
Activity diagram

- Visualizes how a customer registers, logs in, browses, customizes, and orders a game, while the owner can update listings and track orders.
- Involves four key actors: the Customer, Order System, Payment System, and Business Owner/Manager.



Process flow diagrams

- The level 0 DFD details how data flows between processes like ordering, payment, and management.

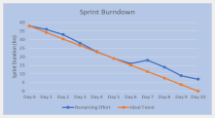


Entity Relationship Diagram (ERD)

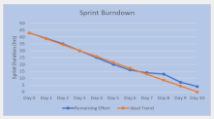
- Shows the relationships between users, orders, games, and feedback, ensuring data is stored and retrieved efficiently.

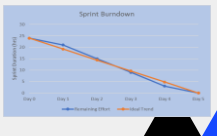
Burn-down chart

- 7 hours were remaining from this sprint.



- 4 hours were remaining from this sprint.

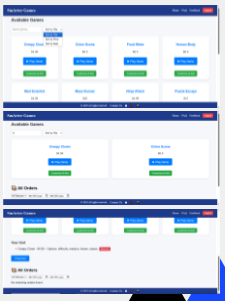




Testing

- Registration

- Filtering, searching, and customization of Games



Conclusion

- Success: learned full-stack, improved teamwork
- Challenges: integrating payment, debugging async code
- Future plans: add real educational escape room games

THANK YOU!